Supporting information to

YR-MPE: a modular desktop platform for agent-operable and reproducible phylogenetic workflows

# YPhylo Workbench Tool Reference

Plugin-by-plugin GUI, CLI and agent reference

Version 1.2.0 — November 10, 2026

Zhijie Xu, Yiyang Jia, Zhihan Gu, Mingrui Shang, Xinbo Cheng, Shuo Han, Yang Jin, Yitong Lei, Rui Zhang, Yunlong Zhang

Department of Biological Sciences and Technology, College of Biological Science and Medical Engineering, Donghua University, Shanghai 201620, China

Department of Cardio-Pulmonary Circulation, Shanghai Pulmonary Hospital, Tongji University School of Medicine, Shanghai 200433, China

Shanghai Center of Systems Biomedicine, Shanghai Jiao Tong University, Shanghai, China

Correspondence: Yunlong Zhang, zhyl@dhu.edu.cn

# Contents

**Preface** **3**

**1 Alignment and trimming** **4**

1.1 MAFFT . . . . . 4

1.2 MACSE2 . . . . . 5

1.3 MUSCLE5 . . . . . 7

1.4 Clustal Omega . . . . . 7

1.5 TrimAl . . . . . 8

1.6 GBlocks . . . . . 10

**2 Sequence and dataset management** **11**

2.1 One-Step OrthoFinder . . . . . 11

2.2 Sequence Viewer . . . . . 12

2.3 SeqMatrix . . . . . 13

2.4 SeqDBG . . . . . 14

2.5 Sequence Concatenator . . . . . 15

2.6 Tree Merger . . . . . 17

**3 Model selection** **18**

3.1 IQ-TREE ModelFinder . . . . . 18

3.2 ModelTest-NG . . . . . 19

3.3 Model Parameter Estimation . . . . . 20

**4 Distance and saturation** **22**

4.1 ML distance calculation . . . . . 22

4.2 Uncorrected p-distance . . . . . 23

4.3 Substitution saturation (BaCoCa C-value) . . . . . 24

4.4 DecentTree . . . . . 25

**5 Phylogenetic inference** **27**

5.1 IQ-TREE 3 . . . 27

5.2 MrBayes . . . 29

5.3 PhyloBayes-MPI . . . 31

5.4 MPBoot . . . 32

**6 Species trees and reconciliation** **34**

6.1 ASTRAL-IV / wASTRAL . . . 34

6.2 CASTER-site . . . 36

6.3 TreeRecs . . . 37

6.4 BEST / MrBayes Multi-Species Coalescent . . . 38

**7 Dating and selection** **39**

7.1 LSD2 . . . 39

7.2 PD-Guide . . . 40

7.3 Codon-based Z-test . . . 41

7.4 Batch dN/dS . . . 43

**8 Workflow execution** **45**

8.1 Workflow Runner . . . 45

**9 Visualization and diagnostics** **47**

9.1 IcyTree . . . 47

9.2 MiniTracer . . . 48

9.3 Robinson-Foulds Comparison . . . 49

9.4 YR-MPE Console . . . 50

**A Tool and menu index** **52**

# Preface

This reference is generated from the agent-facing skill files in `YR_MPE/skills`. Each tool section states what the tool does, how the GUI controls map to command-line options or NEXUS directives, the recommended workflow, and the pitfalls that are easy to hit. Tools that are implemented entirely in Python or in the GUI say so explicitly instead of inventing a command line.

The skill files remain the source of truth. Regenerate this document from the repository root and build it from `docs/`:

```
python docs/tools/build_tool_reference.py
latexmk -lualatex manual/tool-reference.tex
```

# Chapter 1

# Alignment and trimming

## 1.1 MAFFT

*Menu:* ALIGN → Align by... → MAFFT
*Skill id:* `mafft`

### 1.1.1 Purpose

Align nucleotide or protein FASTA files and import each alignment into the workspace.

### 1.1.2 GUI to CLI

`control_bindings()` is the source of truth. The expected mapping is:

| GUI control | CLI option | Notes |
|---|---|---|
| File input | positional | Appended after all options |
| Alignment mode | `--auto`, `--localpair`, `--genafpair`, `--globalpair`, `--retree` | High-accuracy modes also add `--maxiterate 1000` |
| Threads | `--thread` | `-1`/automatic is omitted |
| Direction adjustment | `--adjustdirection` | Detects and reverse-complements sequences that appear reversed; marks them with `_R_` |
| Direction adjustment (accurate) | `--adjustdirectionaccurately` | Slower, more reliable for divergent or long reads; also marks reversed records with `_R_` |
| Advanced penalties | `--op`, `--ep`, `--maxiterate`, `--clustalout`, `--reorder`, ... | Set in Advanced Parameters |
| Output suffix | none | GUI-only; MAFFT writes to stdout and the plugin saves it beside the input |

Verify options with:

```
mafft --help
```

### 1.1.3 Flow

1. Select an alignment/sequence or open `ALIGN -> Align by...-> MAFFT`.
2. Choose mode and threads; run.
3. Import the resulting alignment. When MAFFT was launched from a workspace dataset, each alignment updates the matching item in that same dataset (the dataset itself is not duplicated).
4. Continue with TrimAl/GBlocks, ModelTest-NG or ModelFinder, then RAxML-NG/ IQ-TREE.

### 1.1.4 Pitfalls

- MAFFT writes to stdout; the plugin, not MAFFT's `-o`, creates the output file.
- Batch mode runs one MAFFT process per input file.
- Advanced parameters are only included when they differ from MAFFT defaults.
- With direction adjustment enabled, MAFFT prefixes reverse-complemented records with `_R_`. The plugin strips this prefix from output headers so downstream taxon matching (OrthoFinder/concatenation) is not broken; the sequences themselves remain reverse-complemented. Do not re-add `_R_` when constructing taxon labels.
- Direction adjustment is off by default. Enable it for Sanger amplicons or other data of unknown orientation.

## 1.2 MACSE2

*Menu:* ALIGN → Align by... → MACSE
*Skill id:* `macse`

### 1.2.1 Purpose

Codon-aware alignment of coding sequences with MACSE2 2.07. It preserves reading frames, handles frameshifts/stops, and writes both nucleotide and amino-acid alignments.

### 1.2.2 GUI to CLI

The runner builds:

```
macse -prog alignSequences -seq <input> [options] -out_NT <nt> -out_AA <aa>
macse -prog refineAlignment -align <input> [options] -out_NT <nt> -out_AA <aa>
```

| GUI control | CLI option | Notes |
|---|---|---|
| Mode = Align sequences | `-prog alignSequences -seq PATH` | |
| Mode = Refine existing alignment | `-prog refineAlignment -align PATH` | |
| Genetic code | `-gc_def N` | |
| AA alphabet | `-alphabet_AA NAME` | Default SE_B_8 |
| Score matrix | `-score_matrix NAME` | BLOSUM62, VTML200_BIS, VTML240 |
| Optimization cycles | `-optim N` | Default 2 |
| NT output | `-out_NT PATH` | Plugin-managed |
| AA output | `-out_AA PATH` | Plugin-managed |
| Preferred result | none | GUI-only: selects NT or AA for import |

Advanced parameters are free text and are appended verbatim to the MACSE command after shell-style splitting.

### 1.2.3 Post-processing (GUI-only)

- **Mask exceptional characters** rewrites `!` to `?`.
- **Mask terminal stop codon** replaces a terminal stop with `---`.
- **Drop all-gap codon columns** removes columns where every sequence has a gap.

These are applied by the plugin after MACSE, not by MACSE flags.

### 1.2.4 Flow

1. Import nucleotide FASTA files.
2. Choose align or refine mode.
3. Set genetic code, alphabet, matrix and optimization.
4. Run; send the NT or AA alignment to the workspace.

### 1.2.5 Pitfalls

- Align mode expects unaligned CDS; refine mode expects an existing codon alignment.
- The genetic code is passed by its NCBI number, not the display label.
- MACSE writes both NT and AA files; the preferred-result combo only controls which one is selected for import.
- Free-text advanced parameters are passed through without validation.

## 1.3 MUSCLE5

*Menu:* ALIGN → Align by... → Muscle 5
*Skill id:* `muscle5`

### 1.3.1 Purpose

Fast multiple-sequence alignment with MUSCLE5.

### 1.3.2 GUI to CLI

The runner calls `muscle -align|-super5 <input> -output <output>`.

| GUI control | CLI option | Notes |
|---|---|---|
| Input FASTA | positional | |
| Mode = Standard alignment | `-align` | Default |
| Mode = Large dataset | `-super5` | For many sequences |
| Output | `-output PATH` | Plugin-managed |
| Output suffix | none | GUI-only; default `_m5.fas` beside the input |

### 1.3.3 Flow

1. Import one or more FASTA files.
2. Choose standard or large-dataset mode.
3. Run; the alignment is written beside the input (or to a temporary file) and can be sent to the workspace.

### 1.3.4 Pitfalls

- `-align` and `-super5` are mutually exclusive modes, not tuning flags.
- MUSCLE5 writes through `-output`; the GUI suffix is not a MUSCLE option.
- Batch mode runs one process per input file.
- `-super5` is intended for large datasets and can differ slightly from `-align`; do not mix modes within one comparison.

## 1.4 Clustal Omega

*Menu:* ALIGN → Align by... → Clustal Omega
*Skill id:* `clustal-omega`

### 1.4.1 Purpose

Multiple-sequence alignment with Clustal Omega 1.2.2.

### 1.4.2 GUI to CLI

The runner calls:

```
clustalo -i <input> -o <output> --outfmt=fa [--auto] [--seqtype T] [--threads N] [--outorder=tree-order]
```

| GUI control | CLI option | Notes |
|---|---|---|
| Input FASTA | `-i PATH` | |
| Auto options | `--auto` | Default on |
| Sequence type | `--seqtype Protein\|RNA\|DNA` | Auto is omitted |
| Threads | `--threads N` | Omitted when 1 |
| Output order = tree-order | `--outorder=tree-order` | Input order adds nothing |
| Output | `-o PATH` | Plugin-managed |
| Output format | `--outfmt=fa` | Always FASTA |

### 1.4.3 Flow

1. Import one or more FASTA files.
2. Leave Auto on unless you need explicit guide-tree/HMM control.
3. Set sequence type and threads.
4. Run; send the aligned FASTA to the workspace.

### 1.4.4 Pitfalls

- `--auto` overrides manual guide-tree/HMM settings; turn it off before expecting those options to matter.
- `--seqtype` is case-sensitive in the GUI mapping; Auto means Clustal detects the type itself.
- The output is always FASTA (`--outfmt=fa`); the GUI suffix only changes the filename.
- Batch mode runs one process per input file.

## 1.5 TrimAl

*Menu:* ALIGN → Trim Alignment by... → Trimal
*Skill id:* `trimal`

### 1.5.1 Purpose

Trim gap-rich and ambiguous columns from a multiple-sequence alignment with TrimAl 1.2rev57.

### 1.5.2 GUI to CLI

The runner builds `trimal -in <input> -out <output> [mode] [format]`.

| GUI control | CLI option | Notes |
|---|---|---|
| Input alignment | `-in PATH` | One run per file in batch mode |
| Trimming mode | `-gappyout`, `-strict`, `-strictplus`, `-automated1` | Default: automated1 |
| Output format | `-clustal`, `-nexus`, `-phylip`, `-phylip3.2`, `-mega`, `-nbrf` | FASTA adds no flag |
| Output file | `-out PATH` | Plugin-managed |
| Output suffix | none | GUI-only; output is beside the input |

Advanced parameters (applied only after accepting the dialog):

| GUI control | CLI option |
|---|---|
| Gap threshold | `-gt X` |
| Similarity threshold | `-st X` |
| Consistency threshold | `-ct X` |
| Minimum positions | `-cons N` |
| Window size | `-w N` |
| Gap window | `-gw N` |
| Similarity window | `-sw N` |
| Consistency window | `-cw N` |
| Residue overlap | `-resoverlap X` |
| Sequence overlap | `-seqoverlap N` |
| Remove all-gap columns | `-nogaps` |
| Remove columns composed only of gaps | `-noallgaps` |
| Complementary alignment | `-complementary` |
| Column numbering | `-colnumbering` |

Thresholds are only emitted when greater than zero; the default mode supplies its own internal thresholds.

### 1.5.3 Flow

1. Import an alignment.
2. Choose a trimming mode and output format.
3. Optionally set advanced thresholds/windows.
4. Run; send the trimmed alignment to the workspace.

### 1.5.4 Pitfalls

- `-strict` and `-automated1` are alternatives; only one mode flag is emitted.
- Advanced threshold values of zero are omitted, not passed as `0`.

- `-nogaps` removes any column containing a gap; `-noallgaps` only removes columns made entirely of gaps. They are not interchangeable.
- FASTA output has no format flag; the other formats add one.

## 1.6 GBlocks

*Menu:* ALIGN → Trim Alignment by... → GBlocks
*Skill id:* `gblocks`

### 1.6.1 Purpose

Select conserved alignment blocks with GBlocks 0.91b.

### 1.6.2 GUI to CLI

The runner calls `GBlocks <input> [options] -p=n -q=n -r=n -e=-gb`.

| GUI control | CLI option | Notes |
|---|---|---|
| Input alignment | positional | First argument |
| Sequence type | `-t=p`, `-t=d`, `-t=c` | Auto adds nothing |
| Minimum block length | `-b1=N` | Omitted at 0 |
| Minimum conserved positions | `-b2=N` | Omitted at 0 |
| Max contiguous non-conserved | `-b3=N` | Omitted at 8 |
| Minimum flank positions | `-b4=N` | Omitted at 10 |
| Gap positions | `-b5=n\|h\|a` | None is omitted |
| Similarity matrix | `-b6=y\|n` | Always emitted |
| Output file | `-e=-gb` | Plugin-managed extension |
| Batch/report flags | `-p=n -q=n -r=n` | Always added |

### 1.6.3 Flow

1. Import an alignment.
2. Choose sequence type and block parameters.
3. Run; send the `.gb` alignment to the workspace.

### 1.6.4 Pitfalls

- `-b1`/`-b2` are omitted at 0, not passed as zero; GBlocks then uses its own defaults.
- `-b3` and `-b4` have non-zero defaults (8 and 10); only changed values are emitted.
- `-b6` is always emitted, including `-b6=n` when similarity matrices are off.
- The plugin suppresses interactive prompts with `-p=n -q=n -r=n`.
- GBlocks output is a block-selected alignment, not a tree.

# Chapter 2

# Sequence and dataset management

## 2.1 One-Step OrthoFinder

*Menu:* WORKSPACE → Dataset → One-Step OrthoFinder (for Organelle Genomes)
*Skill id:* `orthofinder`

### 2.1.1 Purpose

Turn GB/GBFF/GFF inputs into a workspace dataset of single-copy orthogroups. The plugin runs OrthoFinder, extracts CDS for each single-copy orthogroup and imports the resulting alignments/sequences into YPhylo Workbench.

### 2.1.2 When to use it

- You have organelle or bacterial genome annotations and want orthologous loci for downstream alignment and phylogenetics.
- You need a dataset that can be sent to MAFFT/MACSE, ModelTest-NG and RAxML-NG.

### 2.1.3 Semantic actions

This plugin is the most complete agent-facing plugin. Use `workbench.list_windows` to get its `window_id`, then `workbench.invoke_plugin` with:

- `orthofinder.inspect_options` - current options and constraints.
- `orthofinder.modify_options` - atomically update options.
- `orthofinder.validate` - validate annotations and the OrthoFinder runtime.
- `orthofinder.run` - start without opening modal dialogs.
- `orthofinder.stop` - stop the running pipeline.
- `orthofinder.inspect_status` - state, progress and capabilities.

- `orthofinder.inspect_console` - cursor window of console lines.
- `orthofinder.inspect_actions` - native action list and `next_import`.
- `orthofinder.preview_results` - generated files and single-copy OGs.
- `orthofinder.export_to_workspace` - idempotent import of the completed run.

### 2.1.4 Typical flow

```
workbench.open_orthofinder
workbench.list_windows -> window_id
orthofinder.inspect_options
orthofinder.modify_options {source_type: "gbff", input_path: "...", output_dir: "...",
genetic_code: 11, threads: 2}
orthofinder.validate
orthofinder.run
orthofinder.inspect_status
orthofinder.preview_results
orthofinder.export_to_workspace
workbench.workspace
```

After import, each single-copy orthogroup is a dataset item and can be sent to the alignment tools.

### 2.1.5 Pitfalls

- Validation must pass before `orthofinder.run`; check `inspect_status`.
- The export action is idempotent; call it again after a completed run instead of duplicating the dataset manually.
- The plugin cannot close while the pipeline is running.

## 2.2 Sequence Viewer

*Menu:* ALIGN → Open Sequence Files
*Skill id:* `sequence-viewer`

### 2.2.1 Purpose

View and edit a multiple-sequence alignment in a spreadsheet-style window, and launch alignment/trimming plugins on the current data.

### 2.2.2 GUI features

- Open FASTA/alignment files.
- Insert sequences from another file.

- Save changes, Save As, and export the current alignment.
- Switch nucleotide/protein colouring and font.
- Launch MAFFT, Clustal Omega, MUSCLE5, TrimAl or GBlocks from the viewer.
- Navigate by sequence and column position.

There is no command-line interface; the viewer also serves as a lightweight editor for alignments produced by the plugins.

### 2.2.3 Workflow

1. Open an alignment (or receive one from a plugin).
2. Inspect/align columns, insert missing sequences if needed.
3. Optionally launch an aligner or trimmer from the toolbar.
4. Save changes or Save As before importing elsewhere.

### 2.2.4 Pitfalls

- Editing is in-memory until Save/Save As; closing without saving loses edits.
- Inserting sequences can change the alignment width; check that all rows stay rectangular.
- The viewer is not a phylogeny tool; use the tree viewers for Newick/NEXUS.
- Launching a plugin from the viewer passes the current alignment, but the returned result is a new item; it does not overwrite the viewer silently.
- Colour schemes are display-only and do not affect the exported sequences.

## 2.3 SeqMatrix

*Menu:* WORKSPACE → Dataset → Create by SeqMatrix
*Skill id:* `seqmatrix`

### 2.3.1 Purpose

Build and edit multi-locus datasets in a spreadsheet-style matrix, including sequence import, NCBI downloads, partition definitions and export.

### 2.3.2 GUI features

- Import from sequences, NEXUS, partitioned NEXUS, or Excel.
- Download sequences from NCBI by accession.
- Append/delete rows and columns.
- Edit sequence names and partition metadata.
- Export partitions as FASTA files or as partitioned NEXUS.
- **Import as a dataset** sends the matrix to the workspace as typed items.

There is no command-line interface; SeqMatrix is a data-management GUI.

### 2.3.3 Workflow

1. Create or import a matrix.
2. Add taxa (rows) and loci/partitions (columns), or download sequences.
3. Edit names/partitions so rows and columns are consistent.
4. Export FASTA/NEXUS, or import as a dataset for alignment/tree plugins.
5. Continue with MAFFT/TrimAl/IQ-TREE from the dataset.

### 2.3.4 Pitfalls

- Rows are taxa and columns are partitions; mixing the two produces a transposed dataset.
- NCBI downloads require network access and valid accessions; downloaded sequences are grouped separately until placed.
- Partition definitions must match the exported FASTA order.
- Importing as a dataset creates workspace items; exporting to files does not.
- Sequence names must be unique within a partition for downstream matching.

## 2.4 SeqDBG

*Menu:* WORKSPACE → Dataset → Create by SeqDBG
*Skill id:* `seqdbg`

### 2.4.1 Purpose

Compare gene order and annotation across organelle genomes, reconcile gene names, inspect adjacency relationships and export a normalized dataset.

### 2.4.2 GUI features

- Load multiple annotated genomes.
- Build a gene-adjacency graph.
- Normalize gene names with synonym rules.
- View left/right adjacency counts per gene.
- Edit and export/import normalization rules.
- Export selected loci as FASTA or as a workspace dataset.

There is no external command-line tool; SeqDBG runs in Python.

### 2.4.3 Workflow

1. Load annotated genomes.
2. Review the gene list and adjacency relationships.
3. Add normalization rules where annotations use different names for the same gene.
4. Refresh the graph/table so adjacency counts use the normalized names.
5. Select loci and export as a dataset for alignment/tree inference.

### 2.4.4 Pitfalls

- Adjacency is computed within each genome; a gene missing from some genomes appears with lower coverage.
- Normalization rules change labels, not the underlying sequences.
- Gene-order comparisons are not orthology calls by themselves; verify one-to-one correspondence before concatenation.
- Exporting to the workspace creates typed items; exporting FASTA only writes files.
- Strand/orientation is not normalized by renaming; check reverse-complemented genes separately.

## 2.5 Sequence Concatenator

*Menu:* WORKSPACE
*Skill id:* `sequence-concatenator`

### 2.5.1 Purpose

Concatenate aligned FASTA partitions into one supermatrix and generate a NEXUS charset file describing the partition coordinates.

### 2.5.2 GUI workflow

1. **Add Files** selects the aligned partition FASTA files.
2. Each file must be internally rectangular: all sequences in that file have the same aligned length.
3. Set the output FASTA path.
4. Choose whether the NEXUS partition text is shown in a dialog or written to a file.
5. **Concatenate Sequences** writes the supermatrix and charset definitions.

There is no external command-line tool; concatenation is performed in Python.

### 2.5.3 Matching and missing data

- Sequences are matched by their **full FASTA description**, not by a separate taxon ID field.
- Descriptions present in any partition are included in the supermatrix.
- A taxon absent from one partition is filled with `?` for that partition's full length.
- Partition lengths come from each input file; a file with ragged sequence lengths is rejected.

### 2.5.4 Output contract

- Output FASTA contains one record per unique description, with concatenated sequence.
- The NEXUS text contains one `charset` per input file, with 1-based coordinates covering the concatenated length.
- File names are sanitised into NEXUS labels (punctuation replaced by `_`).

### 2.5.5 Pitfalls

- Matching by full description means headers must be identical across partitions; `taxonA` and `taxonA description` are different taxa.
- Every input must be aligned; the concatenator does not align or trim.
- Missing data is `?`, not `-` or `N`.
- Partition order follows the file table, so reorder files before running if the charset order matters.
- The NEXUS output describes partitions only; it does not contain the sequence matrix.

## 2.6 Tree Merger

*Menu:* PHYLOGENY
*Skill id:* `tree-merger`

### 2.6.1 Purpose

Collect trees from multiple Newick files into one tree file, usually to prepare a gene-tree set for ASTRAL/wASTRAL or another supertree method.

### 2.6.2 GUI workflow

1. **Add Files** selects one or more Newick files.
2. **Remove Selected** / **Clear All** edit the file list.
3. Set the output file.
4. **Merge Trees** writes every non-empty line from every input file, one tree per line, to the output.

There is no external command-line tool; the merge is performed in Python.

### 2.6.3 Output contract

- Output is plain Newick text, one tree per line.
- Files are processed in the order shown in the file table.
- If a file contains several trees, each non-empty line is appended.
- The merger does not reroot, deduplicate, validate, or relabel trees.

### 2.6.4 Pitfalls

- Lines that are not Newick are copied verbatim; check the output before running ASTRAL.
- A NEXUS tree block is not parsed; use plain Newick files.
- Taxon labels must already be consistent across files.
- The output order follows the table order, so reorder files if the downstream tool depends on locus order.

# Chapter 3

# Model selection

## 3.1 IQ-TREE ModelFinder

*Menu:* MODEL → Find Best Substitution Model (ML)
*Skill id:* `modelfinder`

### 3.1.1 Purpose

Run IQ-TREE ModelFinder (`-m TESTONLY`) and import the best substitution model as a workspace `model` item.

### 3.1.2 GUI to CLI

| GUI control | CLI option | Notes |
|---|---|---|
| File input | `-s` | One IQ-TREE process per alignment (or a supermatrix in partition mode) |
| Sequence type | `-st DNA` or `-st AA` | `AUTO` omits `-st` |
| Criterion | default BIC; `-AICc` / `-AIC` | BIC is IQ-TREE's default |
| Threads | `-nt` | Omitted when 1/AUTO |
| Model set | `-mset ALL` | The plugin searches all models |
| Partition mode | `-p` / `-q` / `-m` scheme | Built by the partition thread |

The core command is `iqtree3 -s <alignment> -m TESTONLY <options>`.

### 3.1.3 Flow

1. Select an alignment or open `MODEL -> Find Best Substitution Model (ML)`.
2. Run and inspect the model table.
3. Import the model item.
4. Select the model and open IQ-TREE or RAxML-NG; the host applies the best model when the naming is compatible.

### 3.1.4 Pitfalls

- ModelFinder's model names use IQ-TREE syntax (`GTR+F+G4`); RAxML-NG accepts most of it but `+FQ` is normalized to `+FE`.
- Partition mode uses a different thread and can take much longer than a single-alignment search.
- In partition mode the supermatrix taxa are the union across all selected loci; taxa missing from a locus are padded with `?`, not dropped.

## 3.2 ModelTest-NG

*Menu:* MODEL → Find Best Substitution Model (ModelTest-NG)
*Skill id:* `modeltest-ng`

### 3.2.1 Purpose

Select a DNA or protein substitution model and import the best model as a workspace `model` item. ModelTest-NG evaluates a candidate set and reports BIC, AIC and AICc in one run.

### 3.2.2 Preconditions

- One or more aligned FASTA/PHYLIP files.
- More than three taxa for reliable model selection.
- For a partitioned dataset, model selection currently runs per alignment; partition mode (`-q`) is not wired yet.

### 3.2.3 GUI to CLI

The plugin's `control_bindings()` is the source of truth. The expected mapping is:

| GUI control | CLI option | Notes |
|---|---|---|
| File input | `-i`, `--input` | One ModelTest-NG process per selected file |
| Sequence type | `-d`, `--datatype` | `DNA -> nt`, `Protein -> aa`; `Auto` is detected from the first record |
| Criterion | none | GUI-only post-processing; ModelTest-NG always reports BIC/AIC/AICc |
| Starting tree | `-t`, `--topology` | `ML starting tree -> ml`, `MP starting tree -> mp` |
| DNA schemes | `-s`, `--schemes` | `3`, `5`, `7`, `11`; only passed for nucleotide data |
| Threads | `-p`, `--processes` | |
| Output prefix | `-o`, `--output` | Generated by the plugin |

| GUI control | CLI option | Notes |
|---|---|---|
| (fixed) | `--force`, `--disable-checkpoint` | Overwrites results; avoids the Cygwin exit crash |

Verify options with:

```
modeltest-ng --help
```

### 3.2.4 Run and import

1. Select an alignment in the workspace, then open `MODEL -> Find Best Substitution Model (ModelTest-NG)`.
2. Choose the criterion used for the exported best model (BIC by default).
3. Run; inspect the criterion tables in Output Preview.
4. Click `Import to Current Platform`. The item data contains `type=model_table`, `best_model`, `criterion` and all three criteria tables.
5. Select the model item in the workspace and open `PHYLOGENY -> Maximum Likelihood (ML) -> RAxML-NG`; the model is applied automatically when possible.

### 3.2.5 Pitfalls

- The BIC/AIC/AICc names are not command-line options; they select which block is exported.
- ModelTest-NG can exit non-zero after writing a valid report; the plugin accepts the report when the `.out` file exists.
- `Auto` is resolved before the command is built; the preview shows `<auto: nt|aa>` when it is not yet resolved.

## 3.3 Model Parameter Estimation

*Menu:* MODEL → Estimate Model Parameters (DNA, ML)
*Skill id:* `model-parameters`

### 3.3.1 Purpose

Estimate substitution-model parameters with IQ-TREE and export the fitted model (Q matrix, base frequencies, rate parameters) as a workspace model item. It can also parse existing `.iqtree` reports without rerunning IQ-TREE.

### 3.3.2 GUI to CLI

`control_bindings()` is the source of truth. The plugin builds a single `-m` string, appending rate suffixes to the base model.

| GUI control | CLI option | Notes |
|---|---|---|
| Input alignment | `-s PATH` | One IQ-TREE process per file in batch mode |
| Substitution model | `-m MODEL` | `auto` becomes `MFP` |
| Discrete Gamma | `+G` | Appended inside `-m` |
| Gamma categories | `+G{n}` | Appended only when +G is checked |
| Invariant sites | `+I` | Appended inside `-m` |
| Empirical frequencies | `+F` | Appended inside `-m` |
| Threads | `-nt N` | Omitted when 1 |
| Load existing `.iqtree` | none | GUI-only parsing path |

The runner always appends `-redo`.

### 3.3.3 Flow

1. Import an alignment or a set of alignments.
2. Choose a model, or keep `auto` (MFP).
3. Set Gamma/+I/+F if needed.
4. Run; inspect the parameter table and send the model to the workspace.

### 3.3.4 Pitfalls

- `+G`, `+I` and `+F` are part of the same `-m` token, not separate options.
- `auto` is a GUI label; the command uses `MFP`.
- Batch mode runs IQ-TREE once per input file, so a shared model is not fitted across loci.
- Loading an existing `.iqtree` file skips estimation and does not probe the sequence data.

# Chapter 4

# Distance and saturation

## 4.1 ML distance calculation

*Menu:* DISTANCE → Compute Pairwise Distances (ML)
*Skill id:* `ml-distance`

### 4.1.1 Purpose

Compute a maximum-likelihood pairwise distance matrix with IQ-TREE and import it as a distance item. The matrix can feed DecentTree or other distance-based tree builders.

### 4.1.2 GUI to CLI

The runner calls:

```
iqtree3 -s alignment [-st DNA|AA] [-m MODEL] [-nt N] -redo
```

| GUI control | CLI option | Notes |
|---|---|---|
| Input alignment | `-s PATH` | One IQ-TREE run per file in batch mode |
| Sequence type | `-st DNA\|AA` | AUTO is omitted |
| Substitution model | `-m MODEL` | `auto` omits `-m` |
| Discrete Gamma | `+G{n}` | Appended to `-m` |
| Invariable sites | `+I` | Appended to `-m` |
| FreeRate | `+R` | Appended to `-m`; category count is not exposed here |
| Ascertainment bias | `+ASC` | Appended to `-m` |
| State frequencies | `+F`, `+FO`, `+FQ` | Estimated adds nothing |
| Threads | `-nt N` | Omitted when 1 |
| Rerun | `-redo` | Always added |

IQ-TREE writes `<alignment>.mldist`; the plugin parses it into a distance matrix item.

### 4.1.3 Flow

1. Import one or more alignments.
2. Choose sequence type and model.
3. Run; inspect the distance table and send it to the workspace.

### 4.1.4 Pitfalls

- `auto` does not mean ModelFinder here: the plugin simply omits `-m` and lets IQ-TREE use its distance default.
- `+R` in this plugin does not take a category count; use the IQ-TREE plugin when you need `+R{n}`.
- The output is a distance matrix, not a tree.
- Batch mode writes one `.mldist` per input.
- Uncorrected p-distances are a separate plugin and do not use IQ-TREE.

## 4.2 Uncorrected p-distance

*Menu:* DISTANCE → Compute Uncorrected Distances (p-distance)
*Skill id:* `uncorrected-distance`

### 4.2.1 Purpose

Compute an uncorrected p-distance matrix from an alignment without a substitution model. This is the observed proportion of differing sites, not an evolutionary distance corrected for multiple hits.

### 4.2.2 GUI controls

| Control | Effect |
|---|---|
| Input alignment | FASTA alignment |
| Sequence type | AUTO, DNA or Protein |
| Gap treatment | Pairwise deletion or complete deletion |
| DNA rule | All substitutions, transitions only, transversions only, or advanced rules |

The calculation is implemented in Python; there is no external command-line tool.

### 4.2.3 Flow

1. Import an alignment.
2. Choose sequence type and gap treatment.
3. For DNA, optionally restrict the counted substitutions.
4. Run; inspect the matrix and send it to the workspace.

### 4.2.4 Output

- Square pairwise p-distance matrix.
- Values range from 0 to 1 (proportion of differing sites).
- The matrix can feed distance-based tree methods, but it is uncorrected.

### 4.2.5 Pitfalls

- p-distance underestimates evolutionary distance when multiple substitutions occur at the same site.
- Pairwise deletion changes the compared length per pair; complete deletion uses one common set of sites.
- Advanced DNA rules are not the same as a transition/transversion ratio model.
- Protein and DNA rules are not interchangeable; set the sequence type first.
- Do not compare p-distances computed with different gap treatments.

## 4.3 Substitution saturation (BaCoCa C-value)

*Menu:* DISTANCE → Calculate & Plot Substitution Saturation
*Skill id:* `saturation`

### 4.3.1 Purpose

Assess substitution saturation in multiple-sequence alignments with the BaCoCa saturation index C. The plugin replaces the earlier simulated saturation analysis with the BaCoCa C-value.

The C-value is used instead of Xia's Iss because Iss is not a clean saturation decision rule (see Xu 2026 preprints). The C-value was introduced by Struck et al. (2008) and implemented in BaCoCa; it has no intrinsic exclusion threshold. The plugin reports it descriptively, with Ti/Tv versus p-distance scatter plots, which is useful for direct inspection when only a few partitions are available.

### 4.3.2 Method

- The plugin computes the BaCoCa saturation index `C = std(Ti/Tv) /std(p)` from pairwise Ti/Tv counts and p-distances.
- Gap treatment can be pairwise deletion or complete deletion.
- Each alignment gets a saturation plot and summary statistics.
- A high C indicates that substitutions are approaching saturation; low C indicates that the alignment still carries phylogenetic signal.

This is an internal Python analysis; no external executable is invoked.

### 4.3.3 GUI workflow

1. Import one or more alignments.
2. Choose gap treatment.
3. Run.
4. Select a file in the result tab to view its plot and statistics.
5. Export the current plot if needed.

### 4.3.4 Output

- Saturation plot per alignment.
- Summary statistics including the C index.
- Per-file selector for multi-file runs.

### 4.3.5 Pitfalls

- The C index depends on the alignment and gap treatment; compare only runs with the same settings.
- Saturation is not the same as model selection: a saturated alignment may still be usable after trimming or with a site-heterogeneous model.
- Pairwise and complete deletion can change the ranking of alignments.
- The plot is descriptive; it is not a formal hypothesis test.

## 4.4 DecentTree

*Menu:* PHYLOGENY → Distance Methods (DecentTree)
*Skill id:* `decenttree`

### 4.4.1 Purpose

Build a distance-based tree from an ML distance matrix with DecentTree's scalable neighbour-joining variants.

### 4.4.2 GUI to CLI

| GUI control | CLI option | Notes |
|---|---|---|
| Input matrix | `-in PATH` | `.mldist` or `.dist` |
| Algorithm | `-t BIONJ-R\|BIONJ-V\|NJ-R-D\|ONJ-R\|ONJ-R-V\|AUCTION` | |
| Output tree | `-out PATH` | Plugin-managed: `<input>.nwk` |
| Banner | `-no-banner` | Always added |

The current Windows build avoids the classic BIONJ/NJ paths, which crash; the exposed algorithms are the verified variants. `BIONJ-R` is the default.

### 4.4.3 Flow

1. Compute or import an ML distance matrix (for example from the ML distance plugin).
2. Choose an algorithm.
3. Run; send the resulting Newick tree to the workspace.

### 4.4.4 Pitfalls

- The input must be a distance matrix, not an alignment.
- DecentTree writes the tree next to the input as `<input>.nwk`; the GUI does not expose a separate output picker.
- Batch mode runs one command per matrix file.
- The algorithm names are case-sensitive CLI tokens; the GUI strips the parenthetical description before passing them.

# Chapter 5

# Phylogenetic inference

## 5.1 IQ-TREE 3

*Menu:* PHYLOGENY → Maximum Likelihood (ML) → IQ-Tree 3
*Skill id:* `iqtree`

### 5.1.1 Purpose

Maximum-likelihood tree inference from an alignment, with optional partition mode, model selection and branch support.

### 5.1.2 GUI to CLI: basic controls

| GUI control | CLI option | Notes |
|---|---|---|
| File input | `-s` | One IQ-TREE process per alignment |
| Sequence type | `-st DNA` / `-st AA` | `AUTO` omits the flag |
| Threads | `-nt` | Omitted when 1/AUTO |
| UFBoot | `-bb N` | Only when the UFBoot checkbox is enabled |
| aBayes | `-abayes` | Boolean flag |
| SH-aLRT | `-alrt N` | Only when the SH-aLRT checkbox is enabled |
| (fixed) | `-redo` | Plugin always asks IQ-TREE to overwrite |

### 5.1.3 Substitution model

The GUI model combo contains the following base models. The live `workbench.help_doc` returns the same catalog under `controls[id=model].choices`, mapping each GUI label to the exact `-m` token.

- **Automatic:** `auto` -> `-m MFP` (ModelFinder plus tree inference).

- **DNA:** JC69/JC, F81, K2P/K80, HKY85/HKY, TNe, TN93/TN, K3P/K81, K81u/K3Pu, TPM2, TPM2u, TPM3, TPM3u, TIM, TIMe, TIM2, TIM2e, TIM3, TIM3e, TVM, TVMe, SYM, GTR.
- **Protein:** Blosum62, cpREV, Dayhoff, DCMut, EAL, ELM, FLAVI, FLU, GTR20, HIVb, HIVw, JTT, JTTDCMut, LG, mtART, mtMAM, mtREV, mtZOA, mtMet, mtVer, mtInv, NQ.bird, NQ.insect, NQ.mammal, NQ.pfam, NQ.plant, NQ.yeast, Poisson, PMB, Q.bird, Q.insect, Q.mammal, Q.pfam, Q.plant, Q.yeast, rtREV, VT, WAG.

The base model is extended into one `-m` string:

| GUI control | Model suffix | Notes |
|---|---|---|
| Rate Heterogeneity = Equal | none | No rate-heterogeneity operation |
| Rate Heterogeneity = Gamma Distribution (+G) | `+G{n}` | `n` from the Rate categories spin box |
| Rate Heterogeneity = FreeRate Model (+R) | `+R{n}` | `n` from the Rate categories spin box (default 4) |
| Invariable Sites | `+I` | Boolean |
| Ascertainment Bias | `+ASC` | Boolean |
| State Frequencies = Estimated | none | ML estimates |
| State Frequencies = Empirical (+F) | `+F` | Empirical |
| State Frequencies = ML-optimized (+FO) | `+FO` | ML-optimized |
| State Frequencies = Equal (+FQ) | `+FQ` | Equal |

Example: `GTR` + Gamma 4 + Invariant + Empirical = `-m GTR+G4+I+F`. Example: `GTR` + FreeRate 8 = `-m GTR+R8`.

### 5.1.4 Partition mode

Partition mode is configured in **Configure Partitions** and uses a partition file generated from the selected dataset:

| GUI partition option | CLI flag | Notes |
|---|---|---|
| Edge-linked Proportional | `-p partition.nex` | Proportional branch lengths |
| Edge-linked Equal | `-q partition.nex` | Equal branch lengths |
| Edge-unlinked | `-Q partition.nex` | Unlinked branch lengths |
| Separate Tree | `-S partition.nex` | Separate tree per partition |
| Enable Bootstrap + replicates | `-bb N` | Partition thread also adds `-abayes` |
| Sampling: SITE / GENE / GENESITE | (reserved) | Current command builder records the choice but does not yet pass a sampling flag |
| RCluster + percentage | `--rcluster N` | Accelerated partition merging; passed when the merge path is used |

The partition thread also passes `-m MFP+MERGE` when no per-partition model is set; if a `charpartition` model is present, models are read from the partition file instead.

Taxa for the supermatrix are the **union across all selected partitions**; a taxon absent from a locus is padded with `?` for that locus length. Do not take the first partition as the taxon list when the dataset has missing data.

**Partition-merger workaround**

IQ-TREE 3.0.1 can abort partition merging with:

```
Fixing branch lengths not supported under specified site rate model
```

This happens inside the partition-merger evaluation, not in the final tree search. RCluster takes a different pair-search path and avoids it. The plugin passes the RCluster percentage when configured; if the first attempt still fails with this message, it retries once with `--rcluster 10`.

### 5.1.5 Flow

1. Select an alignment; optionally select a model item first.
2. Open `PHYLOGENY -> Maximum Likelihood (ML) -> IQ-Tree 3`.
3. Choose model/rate/state-frequency/support options and run.
4. Inspect `cli.preview` in `workbench.help_doc`; it shows the exact command assembled from the current controls.
5. Import the resulting phylogeny or tree set.

## 5.2 MrBayes

*Menu:* PHYLOGENY → Bayesian Inference (BI) → MrBayes3-MPI-BEAGLE
*Skill id:* `mrbayes`

### 5.2.1 Purpose

Bayesian phylogenetic inference with MrBayes 3.2.7a, including partitioned models, mixed protein models, MCMC diagnostics and optional dating priors. The Windows MPI+BEAGLE build is exposed as a separate plugin entry.

### 5.2.2 How the plugin runs

The plugin writes a `begin mrbayes; ...end;` block into a NEXUS file and runs:

```
mb -i input.nex
```

With MPI enabled it runs `mpirun -np N mb -i input.nex`. If the input already contains a MRBAYES block and **Run MrBayes Data Block directly if present** is checked, that block is used instead of generating a new one.

### 5.2.3 GUI to NEXUS

| GUI control | NEXUS directive |
|---|---|
| Data type = DNA | `lset nucmodel=4by4;` |
| Data type = Protein | `lset nucmodel=protein;` |
| DNA substitution number | `lset nst=N;` |
| DNA state frequencies | `prset statefreqpr=dirichlet(1,1,1,1);`, `fixed (equal)`, or `fixed(empirical)` |
| Protein model | `prset aamodelpr=fixed(model);` or `prset aamodelpr=mixed;` |
| Protein state frequencies | `prset statefreqpr=...;` |
| Rate heterogeneity | `lset rates=equal\|gamma\|invgamma\|propinv\|lnorm\| adgamma;` |
| Gamma categories | `ngammacat=N` in the `lset rates=...` line |
| MCMC settings | `mcmcp ngen=N samplefreq=N nchains=N nruns=N printfreq=N savebrlens=yes checkpoint=yes checkfreq=5000;` then `mcmc;` |
| Consensus type | `sumt contype=majorityrule\|halfcompat\| allcompat;` |
| Consensus format | `conformat=simple\|figtree` |
| Burn-in as fraction | `sump relburnin=yes burninfrac=X;` and `sumt relburnin=yes burninfrac=X;` |
| Burn-in as states | `sump burnin=N;` and `sumt burnin=N;` |
| Partition mode | `charset`/`partition` definitions in NEXUS, then `prset applyto=(all)` as appropriate |
| Dating | `prset brlenspr=clock:uniform;` plus clock and calibration priors |
| BEAGLE | `set usebeagle=yes beagledevice=... beagleprecision=...beaglescaling=...;` |

### 5.2.4 Flow

1. Import a NEXUS/FASTA alignment, or a NEXUS file with an existing MRBAYES block.
2. Set data type, model, rate heterogeneity, MCMC length and consensus options.
3. For partitioned data, enable partition mode and configure partitions before running.
4. Run; inspect the `.con.tre`, `.p` and `.mcmc` outputs; import the consensus tree.

### 5.2.5 Pitfalls

- The generated block must contain `mcmc;`; without it MrBayes only reads the data.
- `sump` and `sumt` must run after `mcmc`; the plugin appends them automatically.
- Burn-in is either a fraction (`relburnin=yes burninfrac=`) or a state count (`burnin=`), never both.
- Partition definitions must live inside the NEXUS file; GUI partition mode only writes them when configured.
- The supermatrix uses the union of taxa across partitions; missing taxa are padded with `?` for that partition, not removed.

- An existing MRBAYES block is not silently replaced: uncheck the direct-run option if the GUI settings should regenerate it.
- The MPI entry needs `mpirun` in addition to `mb`; the standard entry runs the serial executable.

## 5.3 PhyloBayes-MPI

*Menu:* PHYLOGENY → Bayesian Inference (BI) → PhyloBayes-MPI
*Skill id:* `phylobayes`

### 5.3.1 Purpose

Bayesian phylogenetic inference with PhyloBayes-MPI, including CAT mixture models, discrete Gamma rates and site-heterogeneous analyses.

### 5.3.2 GUI to CLI

The runner builds an `mpirun` command:

```
mpirun -np N pb_mpi -d alignment [model options] chain_name
```

| GUI control | CLI option | Notes |
|---|---|---|
| Alignment | `-d PATH` | PHYLIP input |
| MPI processes | `mpirun -np N` | |
| Parallel chains | none | GUI-only: launches N independent chains |
| CAT mixture | `-cat` / `-ncat N` | Infinite or finite CAT |
| nCAT components | `-ncat N` | Enabled by the CAT checkbox |
| Discrete Gamma | `-dgam N` | |
| Substitution model | `-gtr`, `-poisson`, `-jtt`, `-wag`, `-lg` | |
| Sampling frequency + generations | `-x FREQ GEN` | Emitted together |
| Chain name | positional | Plugin-generated from the input name |

Chain post-processing uses `bpcomp`:

| GUI control | bpcomp option |
|---|---|
| Consensus cutoff | `-c X` (0.5 default) |
| Burn-in | `-x BURNIN` |
| Sample every | `-x BURNIN EVERY` |
| Stop at | `-x BURNIN EVERY UNTIL` |

### 5.3.3 Flow

1. Import a PHYLIP alignment.
2. Set MPI processes, CAT/Gamma and substitution model.

3. Run one or more chains.

4. Select the chain files and run `bpcomp` to obtain the consensus tree.

### 5.3.4 Pitfalls

- CAT is enabled by default (`-cat`); switch to finite `-ncat` explicitly.
- Gamma and CAT can be combined; both checkboxes are on by default.
- The MCMC generation control is combined with the sampling frequency into a single `-x` pair, not two separate flags.
- `-1` generations is converted to a large finite value for the CLI.
- Chain analysis uses `bpcomp`, not `pb_mpi`; its cutoff/burn-in options are separate from the MCMC flags.
- PhyloBayes requires PHYLIP input; convert FASTA first.

## 5.4 MPBoot

*Menu:* PHYLOGENY → Maximum Parsimony (MPBoot)
*Skill id:* `mpboot`

### 5.4.1 Purpose

Maximum-parsimony tree inference with MPBoot's ultrafast bootstrap and consensus reconstruction.

### 5.4.2 GUI to CLI

`control_bindings()` covers the top-level controls. Advanced Parameters are collected in a dialog; their flags are listed below.

| GUI control | CLI option | Notes |
|---|---|---|
| Input alignment | `-s PATH` | One MPBoot run per file in batch mode |
| Sequence type | `-st DNA\|AA\|CODON\|BIN\|MORPH` | AUTO is omitted |
| Threads | `-nt N` | Omitted when 1 |
| Ultrafast bootstrap | `-bb N` | Enabled by the UFBoot checkbox |
| UFBoot max iterations | `-nm N` | |
| UFBoot stopping rule | `-nstep N` | |
| UFBoot min correlation | `-bcor X` | |
| RELL epsilon | `-beps X` | |
| Consensus threshold | `-t X` | Omitted at 0 (extended consensus); 0.5 = majority-rule |
| Consensus burn-in | `-bi N` | Omitted at 0 |
| Consensus tree | `-con` | Always appended by the plugin |

Advanced Parameters (dialog values are applied when the dialog is accepted):

| GUI control | CLI option |
|---|---|
| Initial trees | `-numpars N` |
| Best trees | `-toppars N` |
| Candidate trees | `-numcand N` |
| Iterations = fixed | `-n N` |
| Perturbation strength | `-pers X` |
| Method = IQP | `-iqp` |
| Method = IQPNNI (old) | `-iqpnni` |
| Ratchet iterations | `-ratchet_iter N` |
| Ratchet weight | `-ratchet_wgt N` |
| Ratchet percent | `-ratchet_percent N` |
| Turn off Ratchet | `-ratchet_off` |
| SPR radius | `-spr_rad N` |
| Candidate cutoff | `-cand_cutoff N` |
| Turn off tree refinement | `-opt_btree_off` |
| Use NNI instead of SPR | `-nni_pars` |

### 5.4.3 Flow

1. Import an alignment.
2. Choose sequence type and threads.
3. Enable UFBoot and set the replicate/tuning values if needed.
4. Set the consensus threshold; run.
5. Send the `.treefile`/`.contree` result to the workspace.

### 5.4.4 Pitfalls

- `-con` is always emitted, so a consensus tree is produced even when the threshold is 0.
- `-bb` should be at least 1000; the GUI exposes the tuning options separately.
- Advanced dialog values only take effect after accepting the dialog.
- MPBoot uses the input alignment name for output; batch mode therefore writes one `.treefile`/`.contree` per input.

# Chapter 6

# Species trees and reconciliation

## 6.1 ASTRAL-IV / wASTRAL

*Menu:* SPECIES TREE → ASTRAL-IV
*Skill id:* `astral`

### 6.1.1 Purpose

Infer a species tree from a set of unrooted gene trees. YR-MPE wraps the ASTER executables:

- **ASTRAL-IV** (`astral4`) — standard quartet-based species-tree inference with CASTLES-II branch lengths.
- **wASTRAL** (`wastral`) — weighted ASTRAL; weights quartets by gene-tree support/length and reports coalescent-unit branch lengths.
- **ASTRAL-Hybrid** (`astral-hybrid`) — a synonym/preset of wASTRAL using support values and branch lengths; not a separate method.

### 6.1.2 Citation

- ASTER package: `zhang2025aster`.
- wASTRAL weighting: `zhang2022wastral` (Zhang & Mirarab, 2022).

### 6.1.3 GUI to CLI

The first argv token selects the executable; the remaining options are passed to it. `control_bindings()` is the source of truth.

| GUI control | CLI option | Notes |
|---|---|---|
| Input gene trees | positional | Appended last |
| ASTRAL version | `astral4`, `wastral`, `astral-hybrid` | First argv token |

| GUI control | CLI option | Notes |
|---|---|---|
| Threads | `-t N` | |
| Output tree | `-o PATH` | Optional; a temporary `.tre` is used if empty |
| Support option | `-u 0..3` | 0 no support, 1 length+support, 2 detailed, 3 freqQuad.csv |
| Constraint tree | `-c PATH` | Advanced |
| Guide trees | `-g PATH` | Advanced |
| Rounds | `-r N` | Advanced; default 4 |
| Subsample rounds | `-s N` | Advanced; default 4 |
| Gene-to-taxon map | `-a PATH` | Advanced |
| Root taxon | `--root NAME` | Advanced |
| Subsample proportion | `--proportion X` | Advanced; default 0.25 |
| Seed | `--seed N` | Advanced; default 233 |
| Yule lambda | `-l X` | Advanced; default 0.5 |
| Downweight repeats | `-w X` | Advanced; default 1.0 |
| Verbose level | `-v 1\|2` | Advanced; default 1 |
| Scoring only | `-C` | |
| More rounds | `-R` | |
| Branch lengths (ASTRAL-IV) | `--length SULength\|CULength` | ASTRAL-IV only |
| Gene length (ASTRAL-IV) | `--genelength X` | ASTRAL-IV only |
| Hybrid preset | `-B`, `-S`, `-L` | ASTRAL-Hybrid only |
| Tree weights | `--treeweights PATH` | ASTRAL-Hybrid only |
| Weighting mode | `--mode 1..4` | ASTRAL-Hybrid only |
| Min/max/default support | `-n`, `-x`, `-d` | ASTRAL-Hybrid only |

### 6.1.4 Flow

1. Provide one Newick file with one gene tree per line. Opening ASTRAL from Selected with Phylogeny/TreeSet objects preloads their trees; the **Import from Selected TreeSet** button is the fallback and also accepts plain phylogenies.
2. Choose ASTRAL-IV, wASTRAL or ASTRAL-Hybrid.
3. Set support option, threads and output path.
4. Run; send the resulting tree to the workspace.

### 6.1.5 Weighted-mode support labels

ASTER's weighted parsers (`wastral`, `astral-hybrid`) call `std::stod` on node labels. IQ-TREE can write composite labels such as `/1/100` (SH-aLRT/aBayes/UFBoot). The leading slash makes `std::stod` throw `std::invalid_argument` and the process aborts.

YR-MPE normalizes composite labels before weighted runs: it keeps the first numeric field (aBayes when present, otherwise aLRT/UFBoot) and drops the others. A single support type such as aBayes-only also avoids the crash. ASTRAL-IV accepts composite labels directly.

### 6.1.6 Pitfalls

- ASTER writes the tree to `-o`; stdout contains only logs.

- wASTRAL leaves terminal branch lengths empty in coalescent units; the plugin adds display-only `1e-5` terminal lengths for viewers.
- `--length` and `--genelength` are ASTRAL-IV options; do not pass them to wASTRAL or ASTRAL-Hybrid.
- Constraint and guide trees must be binary Newick trees.

## 6.2 CASTER-site

*Menu:* SPECIES TREE → CASTER-site
*Skill id:* `caster-site`

### 6.2.1 Purpose

Infer a species tree directly from a multiple-sequence alignment by sampling quartets from alignment sites. CASTER-site does not use gene trees, so it is an alternative to ASTRAL when site-level information should drive the estimate.

### 6.2.2 GUI to CLI

`control_bindings()` is the source of truth.

| GUI control | CLI option | Notes |
|---|---|---|
| Input alignment | positional | FASTA, PHYLIP, or a file list |
| Format | `--format fasta\|list\|phylip` | `auto` is omitted |
| Threads | `--thread N` | |
| Output tree | `--output PATH` | Optional |
| Support option | `--support 0..2` | 0 none, 1 length+support, 2 detailed |
| Subsample rounds | `--subsample N` | Advanced; default 4 |
| Constraint tree | `--constraint PATH` | Advanced |
| Guide trees | `--guide PATH` | Advanced |
| Rounds | `--round N` | Advanced; default 4 |
| Gene-to-taxon map | `--mapping PATH` | Advanced |
| Root taxon | `--root NAME` | Advanced |
| Subsample proportion | `--proportion X` | Advanced; default 0.25 |
| Seed | `--seed N` | Advanced; default 233 |
| Chunk size | `--chunk N` | Advanced; default 10000 |
| Ambiguity handling | `--ambiguity 0\|1` | 0=N, 1=diploid unphased |
| Branch lengths | `--length SULength\|CULength` | Advanced; default SULength |
| Scoring only | `--scoring` | |
| More rounds | `--moreround` | |

### 6.2.3 Flow

1. Import one alignment, or a dataset that concatenates to one supermatrix.
2. Set format (usually `auto` or `fasta`), threads and support option.

3. Run and send the tree to the workspace.

### 6.2.4 Pitfalls

- The input is an alignment, not a gene-tree file.
- `--format list` expects a file containing paths to per-locus alignments, not a FASTA alignment.
- CASTER-site can be slow on large alignments; increase threads before increasing rounds.
- The output is a Newick tree with branch lengths; terminal lengths may be missing and are added by the plugin for viewers.

## 6.3 TreeRecs

*Menu:* SPECIES TREE → TreeRecs 1.2
*Skill id:* `treerecs`

### 6.3.1 Purpose

Reconcile a gene tree with a species tree, report duplication/loss events and optionally write orthology/paralogy events.

### 6.3.2 GUI to CLI

| GUI control | CLI option | Notes |
|---|---|---|
| Gene tree | `-g PATH` | Newick |
| Species tree | `-s PATH` | Newick |
| Output directory | `-o DIR` | Plugin-managed |
| Fix root / quiet | `-f -q` | Always added |
| Duplication cost | `-d N` | Default 2 |
| Loss cost | `-l N` | Default 1 |
| Best root by reconciliation | `-r` | Boolean |
| Contraction threshold | `-t X` | Omitted for `none` |
| Species prefix mode | `-p Y` | Only `Y` is emitted |
| Species separator | `-c CHAR` | Omitted for `_` |
| Orthology/paralogy events | `--fevent` | |
| Gene-to-species mapping | `-S PATH` | |

### 6.3.3 Flow

1. Import or select a gene tree and a species tree.
2. Set duplication/loss costs and optional mapping.
3. Run; inspect the reconciliation and import the event tables/tree.

### 6.3.4 Pitfalls

- TreeRecs 1.2 aborts with `std::stod` when given `-t none`; the plugin leaves the flag out unless a numeric threshold is set.
- Passing `-p N` or `-c _` explicitly switches separator mapping on and breaks leaf names such as `A/B/C`; both flags are only emitted when changed from their defaults.
- `-p` uses a species-prefix convention; for a separate mapping file use `-S` instead.
- The output directory is always created by the plugin; `-f -q` makes the run non-interactive.

## 6.4 BEST / MrBayes Multi-Species Coalescent

*Menu:* SPECIES TREE → BEST / MrBayes MSC
*Skill id:* `best-mrbayes`

BEST is the full-likelihood multispecies-coalescent analysis implemented by MrBayes. YPhylo Workbench exposes it as a block-oriented agent surface:

- `best.inspect_options` returns the current structured options and the canonical MRBAYES block.
- `best.apply_block` accepts a complete MRBAYES block. The form is updated from the parsed values and the response returns the canonical block plus any lines that were not parsed.
- `best.modify_options` sets individual controls directly.
- `best.run` starts MrBayes through the task queue. `best.stop` cancels it.
- `best.export_to_workspace` imports the consensus tree and chain files.

The input must be a NEXUS data file. Partitions may be loaded from charset definitions; the species partition uses MrBayes `speciespartition` syntax. A round-trip response should be checked with `unparsed_lines` before running.

# Chapter 7

# Dating and selection

## 7.1 LSD2

*Menu:* CLOCK → LSD2
*Skill id:* `lsd2`

### 7.1.1 Purpose

Least-squares molecular dating with LSD2 2.4.4, using node calibrations and/or tip dates.

### 7.1.2 GUI to CLI

The runner calls:

```
lsd2 -i tree.nwk -s sequence_length -r l|a [-f 100 -q SD] -d calibrations.txt
```

| GUI control | CLI option | Notes |
|---|---|---|
| Tree file | `-i PATH` | Newick |
| Sequence length mode | none | Disabled uses 1000; Manual uses the spin value; From sequence file computes it |
| Sequence length | `-s N` | Always passed |
| Use chosen root | `-r l` | Requires the tree rooted in IcyTree |
| Estimate the root | `-r a` | Default |
| Confidence intervals | `-f 100` | 100 bootstrap trees |
| Lognormal std | `-q X` | Only with confidence intervals |
| Node calibrations | `-d FILE` | Generated from the table |
| Tip dates | `-d FILE` | Merged into the same calibration file |
| Preferred result | none | GUI-only import choice |

### 7.1.3 Calibrations

- Node calibrations are entered in the calibration table.

- Tip dates are entered through **Configure tip dates**; they are merged into the same `-d` file.
- The plugin writes a platform-specific line ending (CRLF on Windows) because LSD2 is sensitive to it.

### 7.1.4 Flow

1. Load a tree; optionally root it in IcyTree.
2. Set sequence length (or load a sequence file).
3. Add node calibrations and/or tip dates.
4. Enable confidence intervals if needed.
5. Run; import the dated tree.

### 7.1.5 Pitfalls

- **Use chosen root** only takes effect if a rooted tree file is loaded; otherwise LSD2 is run with `-r a`.
- Sequence length is always passed. Disabled does not omit `-s`; the runner substitutes 1000.
- `-f` is fixed at 100 bootstrap trees in this plugin.
- `-q` is only added when confidence intervals are enabled.
- At least one calibration is normally required; the GUI can continue without one only after an explicit warning.

## 7.2 PD-Guide

*Menu:* PHYLOGENY → PD-Guide
*Skill id:* `pdguide`

### 7.2.1 Purpose

Set up molecular-dating calibrations and run a first dating analysis. PD-Guide is a guidance layer over the LSD2 controls, with method slots for other dating engines.

### 7.2.2 Method choices

| Method | Status |
|---|---|
| LSD2 | Implemented |
| IQ-TREE tree inference | Implemented as the tree-preparation step |
| PhyloBayes | Not yet implemented |
| MrBayes | Not yet implemented |

| Method | Status |
|---|---|
| MCMCTree | Not yet implemented |

### 7.2.3 GUI workflow

1. Load a tree, or infer one with IQ-TREE from an alignment.
2. Configure tip dates and node calibrations in the shared dating panel.
3. Optionally export the calibration table as a text file.
4. Select LSD2 and run; the dated tree and `.result` files are returned.
5. Inspect the console and send the dated tree to the workspace.

### 7.2.4 Calibration inputs

- Tip dates: one date per sampled sequence.
- Node calibrations: minimum/maximum or distribution-based bounds on internal nodes.
- Calibration tables are written to a temporary file and passed to LSD2 with `-d`.

### 7.2.5 Pitfalls

- Only LSD2 is currently wired end-to-end; the other method entries report "not implemented" rather than silently running.
- The tree must be rooted before dating; PD-Guide can use a chosen root or ask LSD2 to estimate it.
- Calibrations must be compatible with the tree taxa; unmatched labels are not applied.
- Dating requires branch lengths in the input tree.
- Export the calibration table if you need to reuse it with MrBayes/MCMCTree later.

## 7.3 Codon-based Z-test

*Menu:* SELECTION → Codon-based Z-test (NG86 / YN00)
*Skill id:* `codon-ztest`

### 7.3.1 Purpose

Test whether synonymous and nonsynonymous substitution rates differ pairwise among codon-aligned sequences. The plugin implements NG86 (Nei-Gojobori counting) and YN00 (Yang-Nielsen, F84 correction) with an analytic delta-method Z-test.

### 7.3.2 Input

- Codon-aligned nucleotide FASTA.
- All sequences must be in frame and have the same aligned length.
- Stop codons and frameshifts should be masked before running.

### 7.3.3 GUI workflow

1. Select a codon-aligned FASTA.
2. Choose NG86 or YN00.
3. Choose the genetic code.
4. Run.
5. Inspect the pairwise table and the omega matrix; export TSV if needed.

### 7.3.4 Statistics

- Z-test is two-sided by default; one-sided p-values are also reported.
- The upper triangle of the omega matrix reports pairwise omega values; the lower triangle reports the corresponding p-values.
- Significant pairs are highlighted.
- The default variance follows the MEGA-style NG86 implementation; YN00 uses the F84 correction.

### 7.3.5 Output

- Pairwise table: sequence pair, synonymous/nonsynonymous rates, omega, Z, p-values.
- Lower/upper triangular omega/p matrices.
- TSV export.

### 7.3.6 Pitfalls

- The test is pairwise; it does not fit a single tree-wide dN/dS model.
- NG86 can be biased when transition/transversion rates are unequal or divergence is high; YN00 is the F84-corrected alternative.
- Alignment gaps and ambiguous codons reduce the usable codon count.
- A non-significant Z-test does not prove neutrality; it only fails to reject equal rates.
- Genetic code must match the organism; the default is the standard code.

## 7.4 Batch dN/dS

*Menu:* SELECTION → Batch dN/dS (NG86 / YN00 / IQ-TREE)
*Skill id:* `batch-dnds`

### 7.4.1 Purpose

Estimate dN/dS across many codon alignments in one run, using either fast counting methods or IQ-TREE codon-model MLE.

### 7.4.2 Methods

| GUI method | Implementation | Output |
|---|---|---|
| Counting — NG86 | Nei-Gojobori counting | Pairwise omega table |
| Counting — YN00 | Yang-Nielsen F84 correction, one kappa per alignment | Pairwise omega table |
| MLE — IQ-TREE GY | `iqtree -s ALN --seqtype CODON -m GY` | Global omega + logL |
| MLE — IQ-TREE MG | `iqtree -s ALN --seqtype CODON -m MG` | Global omega + logL |

### 7.4.3 GUI workflow

1. Select an input folder of codon-aligned FASTA files.
2. Choose a method and genetic code.
3. For MLE, choose an output folder and threads.
4. Run; inspect the batch table and export TSV.

### 7.4.4 MLE command

```
iqtree -s alignment.fasta --seqtype CODON -m GY|MG -T N --prefix OUT/stem_gy|mg --redo
```

The plugin parses omega and log-likelihood from the IQ-TREE report/stdout.

### 7.4.5 Output

- One row per alignment: method, omega, logL, status, artifact path.
- Failed alignments are marked and kept in the table.
- TSV export of the batch table.

### 7.4.6 Pitfalls

- Counting methods skip stop/ambiguous codons pairwise; MLE requires clean, in-frame codon alignments.
- NG86/YN00 report pairwise omega; GY/MG report one global omega per alignment.
- The MLE output folder is required; picking a file instead of a folder is not supported.
- IQ-TREE must be configured before MLE runs.
- Genetic code applies to both counting and MLE paths.

# Chapter 8

# Workflow execution

## 8.1 Workflow Runner

*Menu:* WORKSPACE → Record & Workflow → Run Workflow...
*Skill id:* `workflow-runner`

### 8.1.1 Purpose

Execute a compiled `yrmpe.workflow.v1` plan. Each step is an ordinary YR-MPE plugin opened in headless mode; the runner applies the step parameters, runs the tool, collects the typed result and feeds the produced files to later steps through placeholder bindings.

### 8.1.2 Input

A JSONL file or an agent transcript. The JSONL form uses one object per line:

```
{"schema": "yrmpe.workflow.v1", "kind": "workflow", "name": "demo",
 "inputs": {"raw": "raw.fas"}}
{"id": "mafft", "op": "mafft.run",
 "outputs": {"alignment": "aln.fas"},
 "params": {"input": "%input.raw%", "output": "%mafft.alignment.fas%"}}
```

- `inputs` maps a role name to an external file. The key becomes `%input.<role><ext>%`.
- A step `outputs` object names each produced artifact. The key becomes `%<step>.<role><ext>%` and is available to later steps.
- `params` keys are the plugin control ids from `control_bindings()`.
- Lines without `op`/`step` are treated as headers or pruned discovery actions.

### 8.1.3 GUI

- **Input & Parameters**: workflow file, output directory and one path field per declared input role.

- **Procedures**: the compiled DAG, one square per tool and one circle per artifact.
- **Output Preview**: one entry per produced file.
- **Console**: each step's command line and tool output.

### 8.1.4 Agent actions

- `workflow.inspect_plan` — compiled steps, inputs and current outputs.
- `workflow.validate` — checks op mapping, input files and output directory.
- `workflow.run` — compile (if needed) and execute.
- `workflow.stop` — stop the current step and the run.
- `workflow.inspect_status` — step count, progress and error.
- `workflow.inspect_console` — console tail.

Generic actions (`plugin.modify_options`, `plugin.run`, `plugin.stop`, `plugin.inspect_console`) also work on this window.

### 8.1.5 Contract

- The runner writes `<output_dir>/workflow_run.json` with the plan, per-step status, command line, produced files and error.
- At the end, all item outputs are merged into one `ResultSpec` (`source="workflow-runner"`) and sent to the workspace.
- A step without `_build_result_spec()` cannot be executed; the runner reports the step as failed instead of inventing an import.

### 8.1.6 Pitfalls

- Placeholder paths in `params` must match an input role or a previous step output; the compiler rejects unknown placeholders.
- The declared output path is a hint. The actual file comes from the step's `ResultSpec`; later steps are rebound to that actual path.
- A step that opens a modal dialog (missing input, missing binary) cannot be used headlessly. Validate inputs and tool configuration first.

# Chapter 9

# Visualization and diagnostics

## 9.1 IcyTree

*Menu:* PHYLOGENY → Tree Viewer (IcyTree)
*Skill id:* `icytree`

### 9.1.1 Purpose

Interactive tree viewing and annotation in an embedded IcyTree web view.

### 9.1.2 Inputs

- Newick (`.nwk`, `.newick`, `.tre`, `.tree`)
- NEXUS (`.nex`, `.nexus`) with a trees block

NEXUS is preferred when the tree carries rich annotations, because IcyTree can display support values, branch lengths and metadata that plain Newick may lose.

### 9.1.3 GUI workflow

1. Open IcyTree from the Phylogeny menu or send a tree/TreeSet to it.
2. The tree is loaded into the embedded viewer.
3. Use IcyTree's own controls to reroot, change layout, colour by support and export.
4. Downloads are offered as SVG, PNG, JPEG, Newick, NEXUS or XML.

There is no command-line interface; IcyTree runs entirely in the embedded web view.

### 9.1.4 Pitfalls

- IcyTree is a viewer, not an inference tool; it does not modify the workspace tree unless you export and re-import it.
- Very large trees may be slow in the embedded browser.
- Export format matters: SVG/PNG for figures, Newick/NEXUS for downstream software.
- Rooting in the viewer is display-side unless the exported tree is re-imported.
- The viewer needs the bundled IcyTree HTML assets; if they are missing, the plugin shows an error page instead of a tree.

## 9.2 MiniTracer

*Menu:* PHYLOGENY → Bayesian Inference (BI) → MCMC Diagnostics (MiniTracer)
*Skill id:* `minitracer`

### 9.2.1 Purpose

Inspect MCMC chain files without an external Tracer installation: trace plots, ESS, 95% HPD, harmonic means, skyline plots and two-chain comparisons. MiniTracer is a Python implementation of Tracer with a redesigned interface layout, not a pixel-for-pixel port of the Java application. Cite Tracer as `rambaut2018posterior` when reporting trace diagnostics.

### 9.2.2 Inputs

- MrBayes `.p` files (tab-delimited chain parameters).
- PhyloBayes `.trace` files.
- Plain tab-delimited trace files with a header row.

### 9.2.3 GUI workflow

1. Open `PHYLOGENY -> Bayesian Inference (BI) -> MCMC Diagnostics (MiniTracer)`.
2. Add one or more chain files; set burn-in.
3. Select parameters in the statistics table (Ctrl/Shift for multiple).
4. Inspect the summary form, box plot or skyline plot.
5. Open **Compare 2 Chains** to select two traces; the scatter panel shows chain 1 against chain 2 with the y = x reference line, and the statistics table reports sample counts, means, the correlation coefficient and the Gelman–Rubin PSRF (R-hat). The harmonic-mean interval plot sits below the right-hand statistics table.

### 9.2.4 Notes for agents

MiniTracer is a GUI analysis tool, not a command-line binary. It consumes chain items from the workspace and produces no new file artifact unless the caller exports one. Use `workbench.workspace` to discover `chain` items and `workbench.selection` to see whether a chain can be sent to MiniTracer.

### 9.2.5 Interpretation

- ESS should be checked before trusting posterior summaries. The statistics table colours rows green at ESS >= 200, yellow at 100–200 and red below 100.
- The two-chain PSRF is a per-parameter Gelman–Rubin R-hat computed from the two post-burn-in traces after truncating to the shorter length; values close to 1 indicate agreement. It is not a multivariate PSRF over the whole model.
- The harmonic-mean panel is descriptive; its approximate delta-method test is not a substitute for a proper marginal-likelihood comparison.
- Skyline plots connect the selected parameters across posterior samples; the dashed bounds are 95% HPD.

## 9.3 Robinson-Foulds Comparison

*Menu:* PHYLOGENY → Tree Comparison → Pairwise Robinson-Foulds (IQ-TREE)
*Skill id:* `rf`

### 9.3.1 Purpose

Compute pairwise unrooted Robinson-Foulds (RF) distances among trees or tree sets with IQ-TREE's `--tree-dist-all`. The plugin exposes the RF matrix, per-pair table, normalized RF and an RF distribution plot.

### 9.3.2 GUI to CLI

| GUI control | CLI option | Notes |
|---|---|---|
| Tree file / TreeSet | `-t PATH` | One Newick tree per line |
| Threads | `-T N` | Default: half the CPU count, capped at 4 |
| Run | `--tree-dist-all` | Always added by the runner |
| Output prefix | `--prefix DIR/rf` | Plugin-managed |
| Rerun | `--redo` | Always added |

Rooted input trees are converted to unrooted topologies in a temporary file before the IQ-TREE call. This normalization is GUI-only and does not appear as a CLI flag.

### 9.3.3 Comparison groups

When the input comes from selected TreeSets, the plugin records which trees belong to the same set. Pairwise rows are labelled `within` or `between` so the box plot can separate within-set and between-set RF distributions. Plain single-tree input produces no group labels.

### 9.3.4 Output

- Lower-triangular RF matrix (TSV and figure).
- Pairwise table: tree 1, tree 2, comparison type, RF, normalized RF.
- Normalized RF uses `RF /(2(n-3))` for `n >= 4` taxa.
- RF distribution box plot.

### 9.3.5 Pitfalls

- RF is unrooted; do not interpret it as a rooted-tree distance.
- The normalized denominator depends on the taxon count; compare normalized values only for the same taxon set.
- Two trees are the minimum input; duplicate splits are counted, not deduplicated.
- IQ-TREE must be configured before Run, even though the dialog opens without it.

## 9.4 YR-MPE Console

*Menu:* WORKSPACE
*Skill id:* `console`

### 9.4.1 Purpose

Open a shell preconfigured with the bundled phylogenetic tools.

### 9.4.2 GUI features

- **Startup a new shell** opens a terminal with the plugin's tool directories added to `PATH`.
- The dialog lists the tool paths and commands that will be available.

There is no analysis command line of its own; the console is a launcher.

### 9.4.3 Workflow

1. Open the Console plugin.
2. Review the tool PATH list.
3. Start a shell.
4. Run bundled tools by name in the opened shell.

### 9.4.4 Pitfalls

- The shell inherits the environment at launch; tools added later require a new shell.
- The console does not create workspace items or provenance records; use the plugins for tracked analyses.
- Closing the dialog does not necessarily close an already-opened terminal.
- Tool paths come from `config.json`; a missing path is shown but cannot be fixed from this dialog.

# Appendix A

# Tool and menu index

| Tool | Menu | Skill id |
|---|---|---|
| One-Step OrthoFinder | WORKSPACE → Dataset → One-Step OrthoFinder (for Organelle Genomes) | `orthofinder` |
| MAFFT | ALIGN → Align by... → MAFFT | `mafft` |
| MACSE2 | ALIGN → Align by... → MACSE | `macse` |
| TrimAl | ALIGN → Trim Alignment by... → Trimal | `trimal` |
| GBlocks | ALIGN → Trim Alignment by... → GBlocks | `gblocks` |
| MUSCLE5 | ALIGN → Align by... → Muscle 5 | `muscle5` |
| Clustal Omega | ALIGN → Align by... → Clustal Omega | `clustal-omega` |
| IQ-TREE ModelFinder | MODEL → Find Best Substitution Model (ML) | `modelfinder` |
| ModelTest-NG | MODEL → Find Best Substitution Model (ModelTest-NG) | `modeltest-ng` |
| Model Parameter Estimation | MODEL → Estimate Model Parameters (DNA, ML) | `model-parameters` |
| IQ-TREE 3 | PHYLOGENY → Maximum Likelihood (ML) → IQ-Tree 3 | `iqtree` |
| RAxML-NG | PHYLOGENY → Maximum Likelihood (ML) → RAxML-NG | `raxml-ng` |
| MrBayes | PHYLOGENY → Bayesian Inference (BI) → MrBayes3-MPI-BEAGLE | `mrbayes` |
| PhyloBayes-MPI | PHYLOGENY → Bayesian Inference (BI) → PhyloBayes-MPI | `phylobayes` |
| MPBoot | PHYLOGENY → Maximum Parsimony (MPBoot) | `mpboot` |
| DecentTree | PHYLOGENY → Distance Methods (DecentTree) | `decenttree` |
| ASTRAL-IV / wASTRAL | SPECIES TREE → ASTRAL-IV | `astral` |
| CASTER-site | SPECIES TREE → CASTER-site | `caster-site` |
| TreeRecs | SPECIES TREE → TreeRecs 1.2 | `treerecs` |
| LSD2 | CLOCK → LSD2 | `lsd2` |
| MiniTracer | PHYLOGENY → Bayesian Inference (BI) → MCMC Diagnostics (MiniTracer) | `minitracer` |

| Tool | Menu | Skill id |
|---|---|---|
| Robinson-Foulds Comparison | PHYLOGENY → Tree Comparison → Pairwise Robinson-Foulds (IQ-TREE) | `rf` |
| Codon-based Z-test | SELECTION → Codon-based Z-test (NG86 / YN00) | `codon-ztest` |
| Batch dN/dS | SELECTION → Batch dN/dS (NG86 / YN00 / IQ-TREE) | `batch-dnds` |
| IcyTree | PHYLOGENY → Tree Viewer (IcyTree) | `icytree` |
| SeqMatrix | WORKSPACE → Dataset → Create by SeqMatrix | `seqmatrix` |
| SeqDBG | WORKSPACE → Dataset → Create by SeqDBG | `seqdbg` |
| Sequence Concatenator | WORKSPACE | `sequence-concatenator` |
| Tree Merger | PHYLOGENY | `tree-merger` |
| BEST / MrBayes Multi-Species Coalescent | SPECIES TREE → BEST / MrBayes MSC | `best-mrbayes` |
| ML distance calculation | DISTANCE → Compute Pairwise Distances (ML) | `ml-distance` |
| Uncorrected p-distance | DISTANCE → Compute Uncorrected Distances (p-distance) | `uncorrected-distance` |
| Substitution saturation (BaCoCa C-value) | DISTANCE → Calculate & Plot Substitution Saturation | `saturation` |
| PD-Guide | PHYLOGENY → PD-Guide | `pdguide` |
| Sequence Viewer | ALIGN → Open Sequence Files | `sequence-viewer` |
| YR-MPE Console | WORKSPACE | `console` |
| Gene Concordance Factors (gCF) | SPECIES TREE → Gene Concordance (gCF) | `gcf` |
| Site Concordance Factors (sCF) | SPECIES TREE → Site Concordance (sCF) | `scf` |
| Site Concordance Diagnostics (sCF + CASTER q1-q3) | SPECIES TREE → Site Diagnostics (sCF + CASTER) | `site-diagnostics` |
| MSC Branch Diagnostics (gCF + ASTER CU) | SPECIES TREE → MSC Diagnostics (gCF + ASTER CU) | `msc-diagnostics` |
| Workflow Runner | WORKSPACE → Record & Workflow → Run Workflow... | `workflow-runner` |